1 Introduction

Dans ce mini-projet vous devez concevoir un système à base de μ-processeur en réutilisant des blocs de composants de base. Les objectifs de ces TP sont multiples. L'objectif premier est de réactualiser vos connaissances sur le langage VHDL, et de les renforcer en introduisant les concepts de réutilisation et de généricité. Cela passe aussi par la maîtrise des outils de simulation du domaine. Nous utiliserons ModelSim dans ces projets, un tutoriel complet est disponible (entre autres) ici : https ://www.microsemi.com/document-portal/doc_view/136363-modelsim-me-10-4c-tutorial-for-libero-soc-v11-7 (se référer principalement au chapitre 3). Enfin une introduction à Verilog est prévue permettant de revoir le concept des architectures de μ-processeurs.

Dans un premier temps vous travaillerez sur le code d'une RAM simple décrite en VHDL, vous la rendrez générique, puis vous créerez un bloc contenant plusieurs instances de cette RAM. Dans un second temps, l'objectif sera d'intégrer ce bloc à un système comprenant également un processeur, implémentant un jeu d'instructions RISC-V, dont le code est écrit en Verilog.

Enfin il faudra simuler l'ensemble afin de démontrer la bonne intégration des différents éléments dans le système.

2 Récupération d'un modèle de RAM

Le point de départ du travail est un modèle simple de mémoire. Ce modèle fonctionnel n'est pas synthétisable mais permet d'émuler une mémoire.

Dans un premier temps récupérez le modèle ramchip.vhd et son testbench sur madoc (Ces fichiers ont été téléchargés à partir du site de la société Doulos :
http ://www.doulos.com/knowhow/vhdl_designers_guide/models/simple_ram_model/)

1. Analyser le code récupéré ainsi que son testbench. Réaliser une simulation complète. Commentez les résultats.
2. Utilisant le type *std_logic_vector()* ce modèle peut poser problème lors d'opérations arithmétiques et il convient donc de remplacer ce type par un type *unsigned()*.
3. Simulez le système pour vérifier que la modification n'a pas changé son fonctionnement.
4. On souhaite rendre ce bloc générique sur la largeur des données et sur le nombre de données mémorisables. Proposez une solution et implémentez-la.
5. Lors de la suite du projet nous réutiliserons ce bloc de base pour créer des mémoires plus grosses en connectant entre elles plusieurs instances de Sync_Ram. Il faut alors ajouter un signal *CS* asynchrone permettant de mettre les sorties de la mémoire à l'état haute impédance (état '*Z*'). Réalisez les modifications et simulez la fonctionnalité de la mémoire incluant ce signal de sélection.

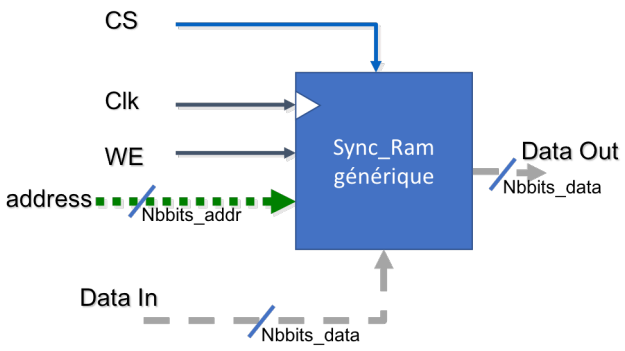


FIGURE 1 – Bloc mémoire générique attendu.

Vous devez, après avoir vérifié par simulation l'ensemble des fonctionnalités, obtenir le bloc Sync_Ram générique comme présenté sur la Figure 1.

3 Réutilisation du bloc Sync_Ram

L'objectif est maintenant de créer un composant mémoire de plus grande capacité contenant N=4 instances de la RAM décrite précédemment. Les signaux d'entrée /sortie sont communs aux quatre blocs, sauf le signal '*CS*' qui doit maintenant être généré par un décodeur prenant en entrée les bits de poids fort du bus d'adresse, qui ont été rajoutés à cet effet. Le système attendu pour N=4 est représenté sur la Figure 2.

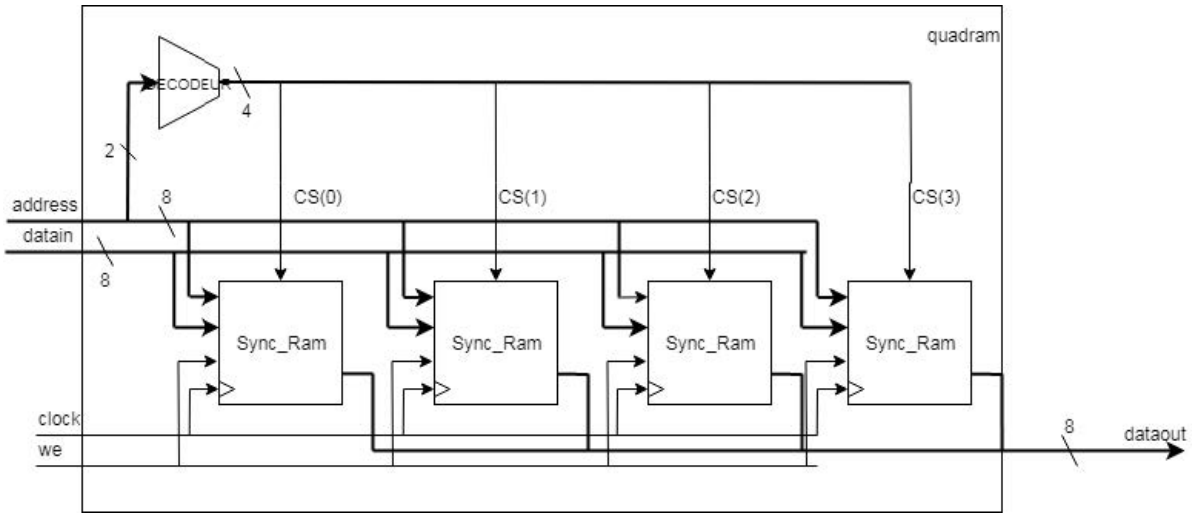


FIGURE 2 – Bloc mémoire final instanciant N=4 blocs Sync_Ram. On remarque que puisque N=4 le décodeur d'adresse utilise 2 bits en entrée.

1. La première étape consiste à créer un décodeur d'adresses. Simuler ce bloc avant son utilisation.
2. Créer le "*top entity*" *quadram* qui :
   (a) inclue 4 blocs Sync_Ram et votre décodeur d'adresses,
   (b) Valider cet ensemble.

3. Rendre le bloc *quadram* générique sur le nombre de Sync_Ram en utilisant la structure *for ... generate* de VHDL et votre décodeur d'adresses qui lui aussi aura été rendu générique.

4 Prise en main d'un modèle de processeur

Une fois que le bloc mémoire est fonctionnel, on souhaite l'utiliser avec un processeur simple récupéré sur le réseau. Ce processeur simple (riscvproc.v) décrit en Verilog est fourni sur Madoc. Ce processeur implémente le jeu d'instructions RISC-V RV32I.

Dans sa version initiale le processeur fonctionne avec une mémoire interne qui sert au stockage du programme et des données. Il ne possède de fait aucun signal d'entrée/sortie, c'est donc un modèle purement simulable (pas de synthèse possible). On souhaite modifier cela et utiliser notre bloc *quadram* pour le stockage des données (i.e. le programme reste dans la mémoire du processeur). Le système final souhaité est représenté sur la Fig. 3.

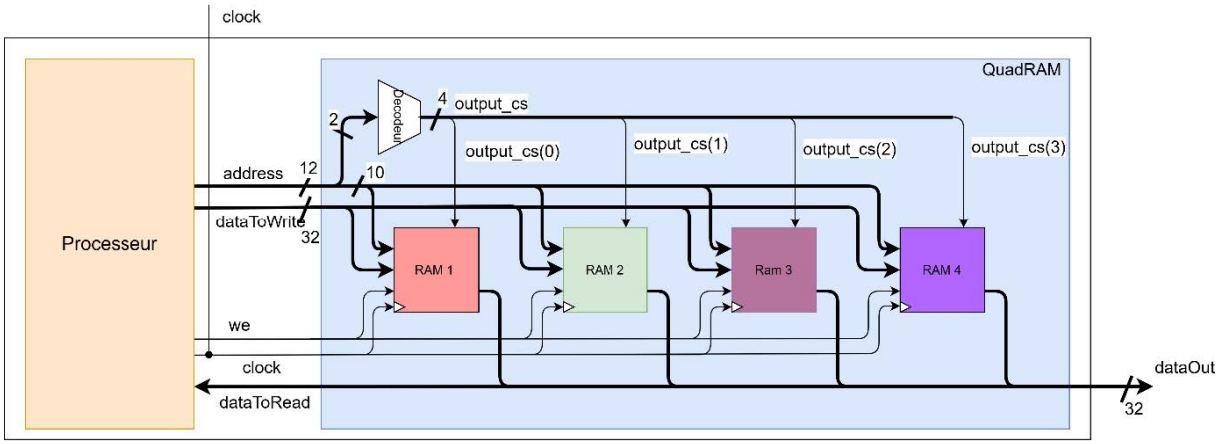


FIGURE 3 – Interfacage du processeur simple avec le bloc quadram précédent.

Il faut donc modifier le processeur afin de pouvoir utiliser le bloc mémoire externe pour le stockage des données. Ces modifications sont de deux ordres :

1. Modifier le jeu d'instructions pour gérer la mémoire externe.
2. Modifier la μ-architecture du processeur afin de pouvoir l'interfacer avec le bloc *quadram*

4.1 Introduction au jeu d'instructions RISC-V

Le jeu d'instructions RISC-V est né en 2010 à l'Université de Californie (UC) Berkeley. Développé au départ en tant que sujet de recherche, le RISC-V est devenu un standard d'ISA (Instruction Set Architecture) ouverte et libre. Porté aujourd'hui par la fondation RISC-V réunissant de nombreux acteurs des technologies micro-électroniques (citons à titre d'exemples Google, Nvidia, IBM, Thales, l'UC Berkeley, l'ETH Zürich), le développement du RISC-V comme jeu d'instructions ouvert permet d'envisager une accélération et des progrès dans le développement des processeurs.

Par opposition au CISC (Complex Instruction-Set Computer), le RISC-V (Reduced Instruction-Set Computer 5th Generation) propose un jeu d'instructions plus simple

favorisant une exécution plus rapide et moins énergivore. Ce jeu d'instructions est également modulable par le biais de plusieurs extensions (extension M pour les multiplications et divisions entières, C pour les instructions compressées, etc...). Le RISC-V supporte plusieurs tailles de mots : 32, 64 et 128 bit, et comme d'autres ISA, il inclut un support multicoeurs et multiprocesseurs.

4.1.1 Manipuler les instructions RISC-V

Une des particularités du RISC-V est la longueur des instructions : celles-ci ont toutes une taille de 32 bit. Partant de ça, reste à savoir comment sont interprétés ces 32 bit ou encore quels sont les *formats d'instructions* ?

Prenons l'exemple d'une addition : on va chercher à additionner le contenu des registres x20 et x21 et placer le résultat dans le registre x9, soit effectuer :

$$x9 = x20 + x21$$

Le code assembleur pour une telle opération est :

add x9, x20, x21

L'instruction *add* nécessitant 3 registres d'entrée, est du type R (*register*). Ainsi d'après la spécification RISC-V, son format est celui présenté par la figure 4.

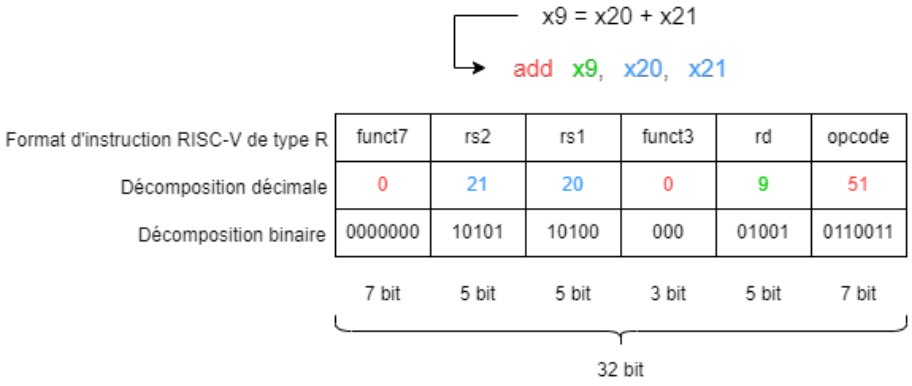


FIGURE 4 – Format de l'instruction "add"

4.1.2 Adapter et implémenter les instructions RISC-V

En consultant la spécification : *RISC-V Unprivileged Specifications Volume I*, `https://riscv.org/technical/specifications/`, et en vous aidant des commentaires dans le code source du processeur, il convient d'adapter le processeur à notre cas d'usage.

1. Les données étant maintenant stockées dans une mémoire externe les instructions ne peuvent plus y accéder directement. Il convient donc de modifier notamment les instructions de manipulation des données pour gérer cette mémoire externe.
2. Le bloc mémoire utilisé est spécifique et il faudra donc veiller au temps de synchronisation requis par le bloc *quadram*.
3. Le processeur fourni supporte le jeu d'instructions RV32I, expliquer quelles sont les modifications à réaliser permettant de supporter le jeu d'instructions RV32IM.

4. En fonction du temps disponible implémenter ces modifications.
   On ne gérera ici que le mode d'adressage direct. On pourra envisager des instructions en adressage immédiat pour l'initialisation de la mémoire.

4.2 Adapter la μ-architecture du processeur RISC-V

Modifier la description de l'architecture matérielle du processeur afin de pouvoir l'interfacer avec le bloc *quadram* :

— Ajout des bus et des signaux de contrôle, permettant la connexion avec le bloc quadram.
— Le processeur initial n'a pas d'horloge, et on pourra prévoir en option un signal de reset.
— D'autres modifications du matériel peuvent être nécessaires en fonction de votre stratégie d'implémentation.

5 Création du système

Le système complet final doit intégrer le processeur modifié et le bloc générique *quadram*. Avant de pouvoir simuler le comportement du processeur, il faut créer un bloc qui liera le processeur avec le bloc de RAM générique. Il est rédigé en VHDL, et instancie le processeur Verilog en tant que "*component*" et crée également une instance de la RAM générique. Il est alors nécessaire de définir des signaux internes au bloc afin de relier ces deux composants.

Afin de valider complètement le système il faut faire exécuter un petit programme au processeur. Il est possible d'initialiser la mémoire interne du processeur avec un fichier qui contiendra les codes héxadécimaux du programme. Ce fichier, d'extension .h, doit être placé dans le répertoire du projet et chargé dans la mémoire du processeur grâce à une commande ModelSim ($readmemh).

Faire **valider** l'ensemble du projet par un enseignant.

6 Question pour un champion en matériel (optionnelle)

Rendre le modèle du système complet synthétisable sur une cible FPGA Xilinx. Pour cela il conviendra de modifier le modèle de mémoire afin d'instancier les mémoires physiques du FPGA, et modifier le code du processeur afin de le rendre réellement synchrone sur le signal d'horloge.

7 Question pour un champion en logiciel (optionnelle)

Rendre le modèle du processeur compatible avec le flot de compilation RISC-V. Pour cela on visera le compilateur C RISC-V 32-bits *trunk* du site *Compiler explorer* (`https://godbolt.org/`). Il est à noter ici que l'ISA RISC-V considère une architecture de Von neumann, il conviendra d'essayer de respecter cette architecture par la gestion correcte de l'espace d'adressage.